AI-Assisted Commit Messages

Overview

Solo-Git can automatically generate high-quality commit messages using AI. The system uses an **Abacus-first routing architecture** that:

1. Prefers Abacus.AI's RouteLLM for intelligent model selection
2. Falls back to OpenAI or Anthropic if Abacus.AI is unavailable
3. Tracks usage, costs, and performance

Key Features

- **Intelligent Routing**: Automatically selects the best AI provider based on your configuration
- **Smart Fallback**: Seamlessly switches to backup providers if the primary fails
- **Cost Optimization**: Uses Abacus.AI's RouteLLM to automatically select the most cost-effective model
- **Conventional Commits**: Supports Conventional Commits format by default
- **Telemetry**: Tracks provider usage, costs, and performance metrics

---

Quick Start

1. Configure API Keys

Edit `~/.sologit/config.yaml`:

```yaml
api_keys:
  abacus_api_key: ${ABACUS_API_KEY}     # Primary provider
  openai_api_key: ${OPENAI_API_KEY}     # Fallback (optional)
  anthropic_api_key: ${ANTHROPIC_API_KEY}  # Fallback (optional)
```

Environment Variables (Alternative):

```bash
export ABACUS_API_KEY="your-key-here"
export OPENAI_API_KEY="your-key-here"  # Optional
export ANTHROPIC_API_KEY="your-key-here"  # Optional
```

2. Generate Commit Message

```bash
# Generate and edit message
evogitctl commit-msg -w my-feature

# Generate without editing
evogitctl commit-msg -w my-feature --no-edit

# Use free-form (non-Conventional Commits)
evogitctl commit-msg -w my-feature --free-form
```

Architecture

```
Request → Policy Engine → Abacus.AI (primary)
                          ↓ (on failure)
                          → OpenAI (fallback #1)
                          ↓ (on failure)
                          → Anthropic (fallback #2)
```

How It Works

1. **Policy Engine**: Receives commit message generation request
2. **Provider Selection**: Selects primary provider based on routing strategy
3. **Generation**: Attempts to generate commit message with primary provider
4. **Fallback**: If primary fails, tries fallback providers in order
5. **Telemetry**: Records usage statistics, costs, and performance

Configuration

Basic Configuration

Add to your `~/.sologit/config.yaml`:

```yaml
ai_commit_message:
  enabled: true
  routing_strategy: abacus_first  # Default strategy
  fallback_chain:
    - abacus
    - openai
    - anthropic
  conventional_commits: true
  user_preference: null  # Force a specific provider (optional)
```

Advanced Configuration

```yaml
ai_commit_message:
  enabled: true
  routing_strategy: cost_optimized
  fallback_chain:
    - abacus
    - openai
  conventional_commits: true
  max_retries: 2  # Per provider
  timeout_seconds: 30
  enable_caching: true

  # Provider-specific settings
  provider_configs:
    abacus:
      model: "routellm-auto"  # Auto-select best model
      timeout: 30
      max_retries: 3

    openai:
      model: "gpt-4o-mini"  # Cost-optimized fallback
      timeout: 30
      max_retries: 3

    anthropic:
      model: "claude-3-5-sonnet-20241022"
      timeout: 30
      max_retries: 3
```

Routing Strategies

Strategy	Description	When to Use
`abacus_first`	Use Abacus.AI, fallback to others	Default, recommended
`cost_optimized`	Use cheapest available provider	Budget-constrained
`latency_optimized`	Use fastest available provider	Speed-critical
`user_specified`	Use `user_preference` provider	Manual control

Strategy Examples

Abacus-First (Default)

```yaml
ai_commit_message:
  routing_strategy: abacus_first
  fallback_chain:
    - abacus
    - openai
    - anthropic
```

Behavior: Always tries Abacus.AI first. If unavailable or fails, falls back to OpenAI, then Anthropic.

Cost-Optimized

```yaml
ai_commit_message:
  routing_strategy: cost_optimized
  fallback_chain:
    - abacus  # Usually cheapest
    - openai
```

Behavior: Selects the cheapest available provider. Abacus.AI's RouteLLM typically offers the best cost/performance ratio.

Latency-Optimized

```yaml
ai_commit_message:
  routing_strategy: latency_optimized
  fallback_chain:
    - openai  # Often fastest response
    - abacus
```

Behavior: Prioritizes response speed over cost.

User-Specified

```yaml
ai_commit_message:
  routing_strategy: user_specified
  user_preference: openai
  fallback_chain:
    - openai
    - abacus
```

Behavior: Always uses your preferred provider first.

Telemetry

View Usage Statistics

```
evogitctl telemetry --days 30
```

Example Output:

```
AI Provider Usage (Last 30 days)
================================
Total Requests: 145
Total Cost: $0.2340
Avg Latency: 210ms
Fallback Rate: 5.5%

┌──────────┬──────────┬────────────┐
│ Provider │ Requests │ Percentage │
├──────────┼──────────┼────────────┤
│ abacus   │      137 │      94.5% │
│ openai   │        6 │       4.1% │
│ anthropic│        2 │       1.4% │
└──────────┴──────────┴────────────┘
```

Telemetry Data

The system tracks:

- **Provider Usage**: Which providers are being used and how often
- **Cost Tracking**: Total costs and per-provider breakdown
- **Latency**: Average response times
- **Fallback Rate**: How often fallback providers are needed
- **Success Rate**: Percentage of successful generations

Data Storage

Telemetry data is stored locally in:

```
~/.sologit/telemetry.jsonl
```

This file is append-only and contains one JSON event per line.

Troubleshooting

“No AI providers available”

Possible Causes:
- API keys not configured
- All providers are disabled
- Network connectivity issues

Solutions:
1. Check API keys in `~/.sologit/config.yaml`
2. Verify keys are valid: `evogitctl config check`
3. Test network connectivity
4. Enable at least one provider in configuration

“All AI providers failed”

Possible Causes:
- Invalid API keys
- Rate limits exceeded

- Network timeout
- Provider outages

Solutions:
1. Verify API keys are correct and active
2. Check provider status pages:
- Abacus.AI Status (https://status.abacus.ai)
- OpenAI Status (https://status.openai.com)
- Anthropic Status (https://status.anthropic.com)
3. Wait a few minutes and retry (may be temporary rate limit)
4. Check `~/.sologit/logs/` for detailed error messages

High Fallback Rate

Possible Causes:
- Primary provider experiencing issues
- Insufficient API credits
- Rate limits being hit

Solutions:
1. Review provider health in telemetry: `evogitctl telemetry`
2. Check API credit balance on provider dashboards
3. Consider adjusting fallback chain order
4. Add more providers to configuration

Slow Response Times

Possible Causes:
- Network latency
- Provider load
- Large diffs being processed

Solutions:
1. Use `latency_optimized` routing strategy
2. Consider changing provider order
3. Check network connectivity
4. Reduce diff size if possible

Invalid or Poor Quality Messages

Possible Causes:
- Insufficient context in diff
- Complex changes requiring more detail
- Provider model limitations

Solutions:
1. Add more context to commits before generating
2. Edit generated message before committing
3. Try a different provider (adjust `user_preference`)
4. Use free-form mode if Conventional Commits format is too restrictive

---

Cost Optimization

Understanding Costs

AI commit message generation is very cost-effective:

- **Abacus.AI RouteLLM**: $0.0001-$0.01 per 1K tokens (automatically selects cheapest appropriate model)
- **OpenAI GPT-4o-mini**: ~$0.15 per 1M input tokens, $0.60 per 1M output tokens
- **Anthropic Claude**: ~$3 per 1M input tokens, $15 per 1M output tokens

Typical Usage

- **Commit Message**: ~100-200 tokens per request
- **Cost per Message**: $0.001-$0.01 (less than a penny)
- **Monthly Estimate** (100 commits): $0.10-$1.00

Cost Reduction Tips

1. **Use Abacus.AI**: RouteLLM automatically selects the cheapest model for your needs
2. **Optimize Diffs**: Keep changes focused to reduce token usage
3. **Batch Commits**: Group related changes when possible
4. **Monitor Usage**: Use `evogitctl telemetry` to track costs
5. **Set Budgets**: Use Solo-Git's cost guard features (see Cost Management (./COST_MANAGEMENT.md))

Cost Comparison Example

Based on 100 commits per month:

Provider	Model	Est. Monthly Cost
Abacus.AI	RouteLLM Auto	$0.10 - $0.50
OpenAI	GPT-4o-mini	$0.30 - $0.60
OpenAI	GPT-4o	$2.00 - $4.00
Anthropic	Claude 3.5 Sonnet	$3.00 - $6.00

Privacy & Security

Data Handling

When you generate an AI commit message:

1. **What is sent**: Git diff, workpad title, optional context
2. **Where it goes**: Selected AI provider's API
3. **What happens**: Provider generates commit message
4. **What is stored locally**: Generated message, telemetry data (provider, cost, latency)

Solo-Git Does NOT Store:

- Your diffs
- Your code
- Your commit messages (beyond local Git)

Provider Privacy Policies

Review each provider's privacy policy:

- **Abacus.AI**: Privacy Policy (https://abacus.ai/privacy)
- **OpenAI**: Privacy Policy (https://openai.com/privacy)
- **Anthropic**: Privacy Policy (https://www.anthropic.com/privacy)

Data Retention

According to provider policies (as of 2024):

- **Abacus.AI**: Does not train on customer data
- **OpenAI**: API data not used for training (for most tiers)
- **Anthropic**: Does not train on API requests

Recommendation: Review provider terms of service and privacy policies, especially for sensitive repositories.

Security Best Practices

1. **API Key Management**:
   - Store keys in `~/.sologit/config.yaml` (not in code)
   - Use environment variables for CI/CD
   - Rotate keys periodically
   - Use separate keys for different environments

2. **Sensitive Repositories**:
   - Disable AI features for sensitive repos: `ai_commit_message.enabled: false`
   - Use self-hosted LLM options if available
   - Review generated messages before committing

3. **Audit Trail**:
   - Telemetry logs stored in `~/.sologit/telemetry.jsonl`
   - Review regularly with `evogitctl telemetry`

Examples

Basic Usage

```
# Generate commit message for current workpad
evogitctl commit-msg -w feature-login

# Generated message (Conventional Commits format):
# feat: Add user authentication with JWT tokens
#
# - Implement login endpoint with email/password validation
# - Add JWT token generation and verification middleware
# - Create user session management
```

Free-Form Messages

```
# Generate without Conventional Commits format
evogitctl commit-msg -w bugfix-123 --free-form

# Generated message (free-form):
# Fixed critical bug in payment processing
#
# The issue was caused by incorrect decimal handling
# in currency conversion. Updated to use proper
# decimal arithmetic library.
```

With Custom Context

```
# Provide additional context
evogitctl commit-msg -w feature-api --context "Implements GraphQL API for product
catalog"

# Generated message:
# feat: Implement GraphQL API for product catalog
#
# - Add GraphQL schema for products, categories, and inventory
# - Implement resolvers with proper authorization
# - Add pagination and filtering support
```

Manual Provider Selection

```
# Force use of specific provider (via config)
evogitctl config set ai_commit_message.user_preference openai
evogitctl commit-msg -w my-feature

# Or use environment variable
SOLOGIT_AI_PROVIDER=anthropic evogitctl commit-msg -w my-feature
```

CLI Reference

`evogitctl commit-msg`

Generate AI-powered commit message for a workpad.

Usage:

```
evogitctl commit-msg -w <workpad> [OPTIONS]
```

Options:

Option	Description	Default
`-w, --workpad`	Workpad ID to generate message for	Required
`--no-edit`	Skip opening editor for review	False (opens)
`--free-form`	Use free-form instead of Conventional Commits	False (uses CC)
`--context TEXT`	Additional context to help generation	None
`--provider TEXT`	Force specific provider (abacus/openai/anthropic)	None (auto)
`--dry-run`	Show what would be generated without committing	False

Examples:

```
# Basic usage
evogitctl commit-msg -w feature-branch

# Skip editor
evogitctl commit-msg -w hotfix-123 --no-edit

# Free-form with context
evogitctl commit-msg -w refactor --free-form --context "Restructured database layer"

# Force OpenAI
evogitctl commit-msg -w experiment --provider openai

# Dry run (preview only)
evogitctl commit-msg -w test --dry-run
```

FAQ

Q: Which provider should I use?

A: Start with the default `abacus_first` strategy. Abacus.AI's RouteLLM provides excellent quality at the lowest cost by automatically selecting the optimal model for each request.

Q: Can I use multiple providers?

A: Yes! Configure a fallback chain to ensure high availability:

```
fallback_chain:
  - abacus
  - openai
  - anthropic
```

Q: How do I disable AI commit messages?

A: Set `enabled: false` in your config:

```
ai_commit_message:
  enabled: false
```

Q: Can I customize the prompt?

A: Currently, prompts are built automatically based on your diff and configuration. Custom prompts will be supported in a future release.

Q: What if I don't like the generated message?

A: By default, the generated message opens in your editor for review and modification. You can always edit before committing.

Q: Does this work offline?

A: No, AI commit message generation requires internet connectivity to reach provider APIs. However, you can always write commit messages manually when offline.

Q: How do I get API keys?

A:
- **Abacus.AI**: Sign up (https://abacus.ai/signup) → API Settings → Generate Key
- **OpenAI**: Platform (https://platform.openai.com/) → API Keys
- **Anthropic**: Console (https://console.anthropic.com/) → API Keys

Q: Can I use this in CI/CD?

A: Yes! Use the `--no-edit` flag for non-interactive environments:

```
evogitctl commit-msg -w $WORKPAD_ID --no-edit
```

Make sure to set API keys via environment variables in your CI/CD configuration.

Integration with Solo-Git Workflow

AI commit messages integrate seamlessly with Solo-Git's workpad workflow:

1. **Create Workpad**: `evogitctl pad create feature-name`
2. **Make Changes**: Edit code in workpad
3. **Run Tests**: `evogitctl test run -w feature-name`

4. **Generate Message**: `evogitctl commit-msg -w feature-name`
5. **Review & Commit**: Edit message if needed, commit
6. **Promote**: `evogitctl pad promote feature-name`

Checkpointing

AI-generated messages work with Solo-Git's checkpoint system:

```
# Create checkpoint with AI message
evogitctl pad checkpoint -w feature-name --ai-message

# Message includes checkpoint context automatically
```

Support

Getting Help

- **Documentation**: Check other docs in `docs/` folder
- **Issues**: GitHub Issues (https://github.com/yourusername/Solo-Git/issues)
- **Discussions**: GitHub Discussions (https://github.com/yourusername/Solo-Git/discussions)

Reporting Bugs

When reporting issues with AI commit messages, include:

1. Config snippet (remove API keys!)
2. Command used
3. Error message
4. Telemetry summary: `evogitctl telemetry --days 1`
5. Relevant logs from `~/.sologit/logs/`

Changelog

v1.0.0 (Current)

- Initial release of AI commit message generation
- Support for Abacus.AI, OpenAI, and Anthropic providers
- Intelligent routing with fallback support
- Conventional Commits format support
- Telemetry and cost tracking
- Comprehensive configuration options

Roadmap

Future enhancements planned:

- [ ] Custom prompt templates

- [ ] Multi-language commit messages
- [ ] Self-hosted LLM support
- [ ] Commit message quality scoring
- [ ] Integration with PR descriptions
- [ ] Voice-to-commit message
- [ ] Team-shared prompt libraries

Contributing

Contributions welcome! Areas for improvement:

- Provider adapter implementations
- Routing strategy algorithms
- Prompt engineering
- Documentation
- Test coverage

See CONTRIBUTING.md (../CONTRIBUTING.md) for guidelines.

License